Εργασία-Αναγνώριση Προτύπων & Μηχανική Μάθηση

Διδάσκων: Επικ. Καθ. Παναγιώτης Πετραντωνάκης (ppetrant@ece.auth.gr)
Βοηθός διδασκαλίας: Υπ. Διδ. Στέφανος Παπαδόπουλος (stefpapad@iti.gr)
2025-2026

Μέρος Α: Εκτίμηση Παραμέτρων (MLE)

- Σκοπός: Υπολογισμός Μέσης Τιμής (μ) και Συνδιακύμανσης (Σ).
- Μέθοδος: Maximum Likelihood Estimation (MLE).
- Δεδομένα: dataset1.csv (2 χαρακτηριστικά, 3 κλάσεις).

Θεωρία MLE

Υπολογίζουμε για κάθε κλάση:

- Μέση Τιμή (μ): Κέντρο βάρους.
- Πίνακας Συνδιακύμανσης (Σ): Διασπορά και συσχέτιση.
- Υλοποίηση: Manual (χωρίς έτοιμες βιβλιοθήκες).

Υλοποίηση MLE (Κώδικας)

Υπολογισμός μ και Σ από τα δεδομένα:

```
for c in classes:
    Xc = X[y == c]
    N = Xc.shape[0]
    # Υπολογισμός μέσης τιμής (μ)
    mu = np.sum(Xc, axis=0) / N

    # Υπολογισμός συνδιακύμανσης (Σ)
    Sigma = np.zeros((2, 2))
    for i in range(N):
        diff = (Xc[i] - mu).reshape(2, 1)
        Sigma += diff @ diff.T
    Sigma /= N
```

Συνάρτηση Πυκνότητας Πιθανότητας

Υλοποίηση του μαθηματικού τύπου P(x|ω):

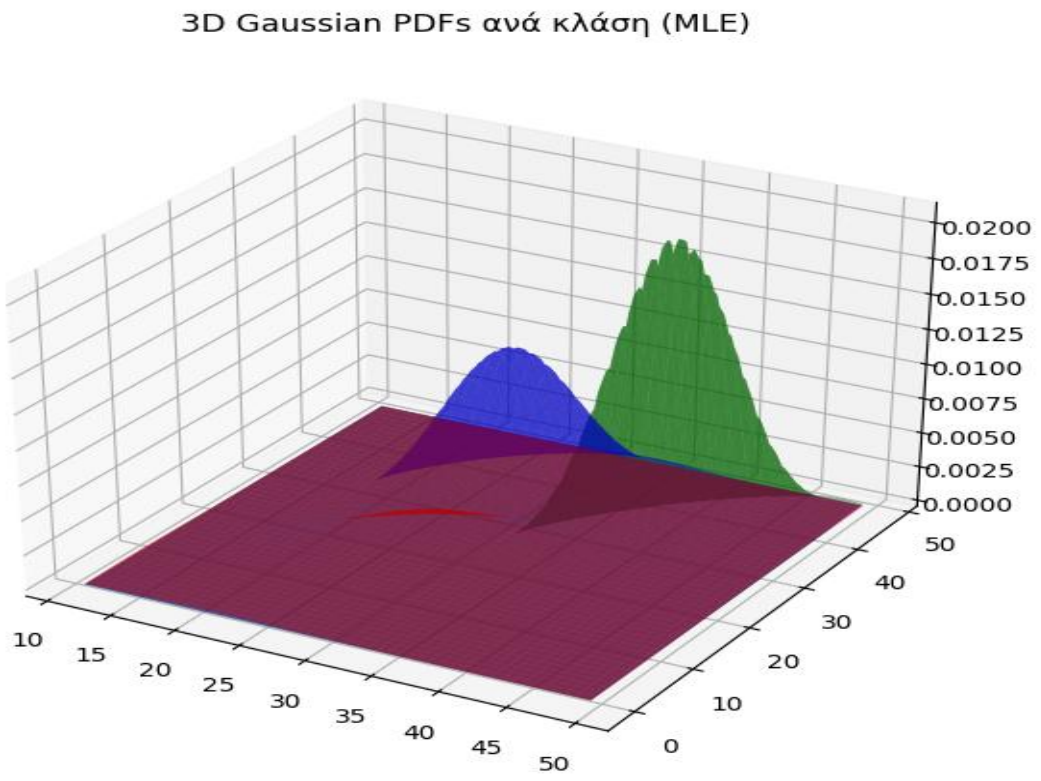
```
def gaussian_2d(X, Y, mu, Sigma):
    ...
    diff = pos - mu
    Z[i, j] = np.exp(-0.5 * diff[i, j] @ Sigma_inv @ diff[i, j].T)
    Z /= (2 * np.pi * np.sqrt(det_Sigma))
    return Z
```

Οπτικοποίηση 3D (Setup)

- Δημιουργία πλέγματος (meshgrid) για σάρωση του χώρου.
- Χρήση Matplotlib 3D projection.
- Κάθε κλάση σχεδιάζεται με διαφορετικό χρώμα.

Γράφημα Κατανομών (3D Plot)

Οπτική επιβεβαίωση των Gaussian Hills.
Δείχνει το διαχωρισμό των κλάσεων στον χώρο.



Αποτελέσματα Παραμέτρων

Οι τιμές που υπολόγισε ο αλγόριθμος.
Σύγκριση των μ και Σ ανά κλάση.

```
Κλάση 0
μ = [29.20903802 16.84988855]
Σ =
 [[46.98591258 22.90532295]
 [22.90532295 48.63133559]]

Κλάση 1
μ = [40.19766916 34.28109099]
Σ =
 [[ 9.42390491 11.48971771]
 [11.48971771 20.10805596]]

Κλάση 2
μ = [27.55232503 34.79318037]
Σ =
 [[13.96874391 11.77433636]
 [11.77433636 25.28848192]]
```

Μέρος Β: Εκτίμηση Πυκνότητας (Parzen)

- Σκοπός: Μη-παραμετρική εκτίμηση πυκνότητας.
- Δεδομένα: dataset2.csv (1D Data).
- Σύγκριση: Με θεωρητική κατανομή N(1, 4).

Θεωρία: Kernels & Window Size

- Kernels (Πυρήνες):
- Uniform: Τετράγωνο παράθυρο (ασυνεχές).
- Gaussian: Καμπάνα (ομαλό).
- Παράμετρος h: Καθορίζει το εύρος του παραθύρου.

Υλοποίηση Parzen (Κώδικας)

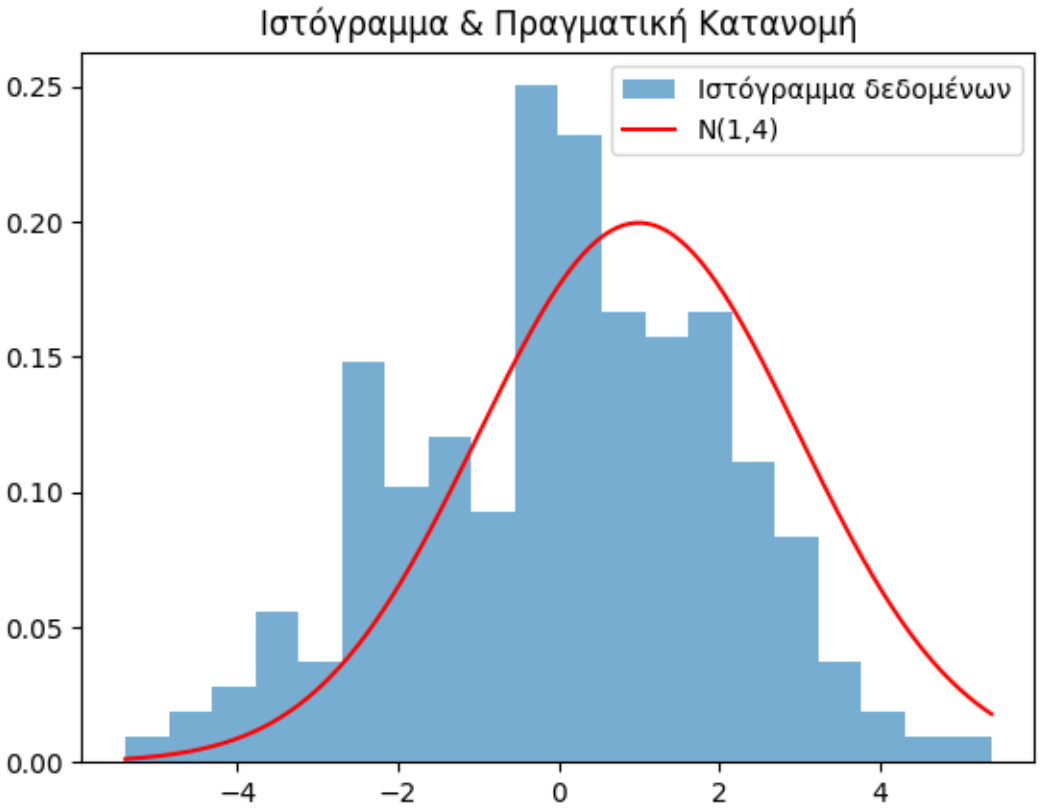
Οι συναρτήσεις των δύο πυρήνων:

```
def parzen_uniform(x, X, h):
    count = np.sum(np.abs(x - X) <= h / 2)
    return count / (N * h)

def parzen_gaussian(x, X, h):
    return np.sum(
        (1 / (sqrt(2 * pi) * h)) *
        np.exp(-((x - X) ** 2) / (2 * h ** 2))
    ) / N
```

Δεδομένα vs Θεωρία

Σύγκριση ιστογράμματος δειγμάτων με την N(1,4).

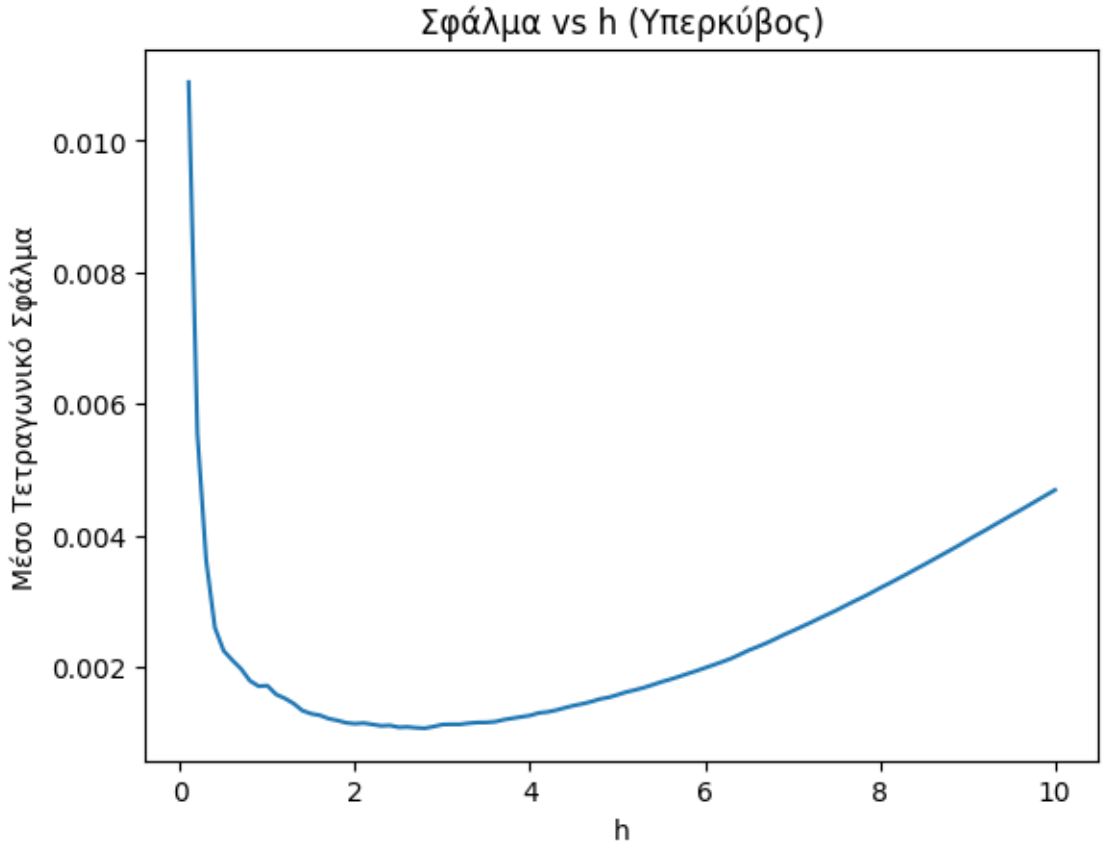


Εύρεση Βέλτιστου h (MSE)

- Υπολογισμός Mean Squared Error (MSE) για διάφορα h.
- Στόχος: Ελαχιστοποίηση της διαφοράς από την πραγματική κατανομή.
- h_values: 0.1 έως 10.1.

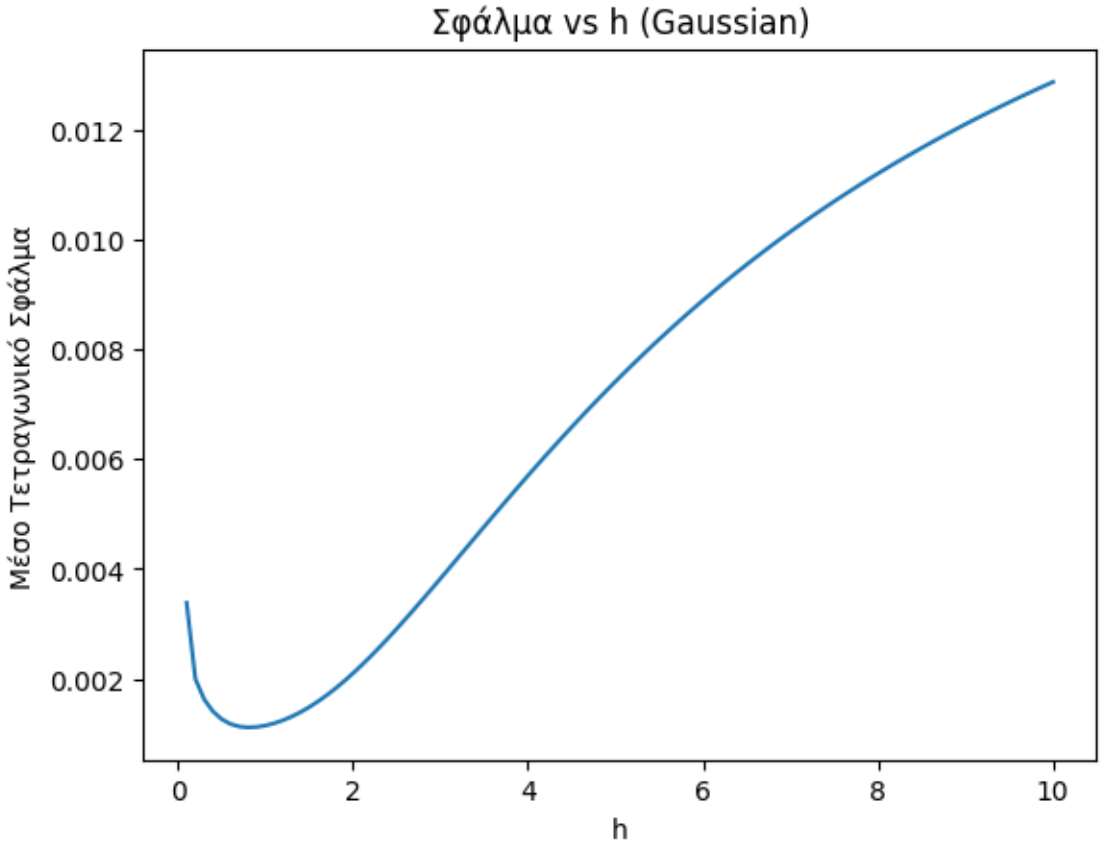
Αποτελέσματα: Uniform Kernel

Διάγραμμα Σφάλματος vs h.
Εντοπισμός του ελάχιστου σημείου (βέλτιστο h).



Αποτελέσματα: Gaussian Kernel

Πιο ομαλή καμπύλη σφάλματος.
Συνήθως δίνει καλύτερη προσέγγιση της φυσικής κατανομής.



Μέρος Γ: k-Nearest Neighbors (k-NN)

- Σκοπός: Ταξινόμηση χωρίς εκπαίδευση (Lazy Learning).
- Δεδομένα: dataset3.csv (Train), testset.csv (Test).
- Αρχή λειτουργίας: Ψήφος των k κοντινότερων γειτόνων.

Βήμα 1: Ευκλείδια Απόσταση

Η βάση του αλγορίθμου.
Υπολογίζει την απόσταση του νέου σημείου από ΟΛΑ τα σημεία εκπαίδευσης.

```
def eucl(x, trainData):
    diff = trainData[:, :2] - x
    distances = np.sqrt(np.sum(diff**2, axis=1))
    return distances
```

Βήμα 2: Εύρεση Γειτόνων

Χρήση argsort για να βρούμε τα indices των μικρότερων αποστάσεων.
Επιλογή των k πλησιέστερων σημείων.

```
def neighbors(x, trainData, k):
    distances = eucl(x, trainData)
    idx = np.argsort(distances) # Ταξινόμηση
    return trainData[idx[:k]]   # Επιστροφή των k πρώτων
```

Βήμα 3: Πρόβλεψη (Voting)

Για κάθε σημείο του Test Set:
1. Βρίσκουμε τους γείτονες.
2. Μετράμε πόσοι ανήκουν στην Class 0 και πόσοι στην Class 1.
3. Επιστρέφουμε πιθανότητες.

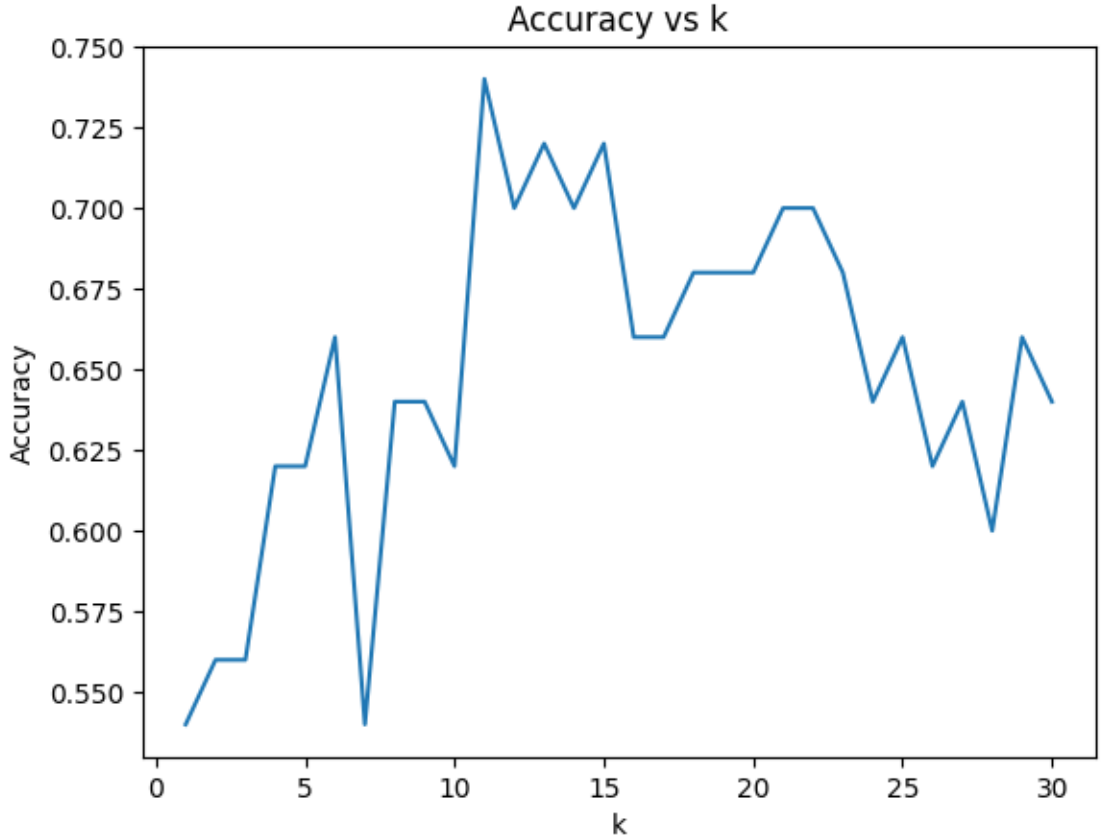
```
def predict(testData, trainData, k):
    ...
    # Υπολογισμός πιθανοτήτων (Voting)
    p0 = np.sum(labels == 0) / k
    p1 = np.sum(labels == 1) / k
    probs.append([p0, p1])
```

Βελτιστοποίηση Παραμέτρου k

- Το k είναι κρίσιμη παράμετρος (Hyperparameter).
- Δοκιμάζουμε τιμές από k=1 έως k=30.
- Υπολογίζουμε την Ακρίβεια (Accuracy) για κάθε k.

Διάγραμμα Accuracy vs k

Γραφική παράσταση της ακρίβειας.
Στόχος: Να βρούμε το k που μεγιστοποιεί την ακρίβεια χωρίς overfitting.
Εμφανίζεται το 'Βέλτιστο k'.



Οπτικοποίηση Ορίων (Code)

Για να δούμε πώς αποφασίζει ο ταξινομητής:
- Δημιουργούμε πυκνό πλέγμα σημείων.
- Κάνουμε πρόβλεψη για ΟΛΟ το πλέγμα.
- Χρησιμοποιούμε contour plot.

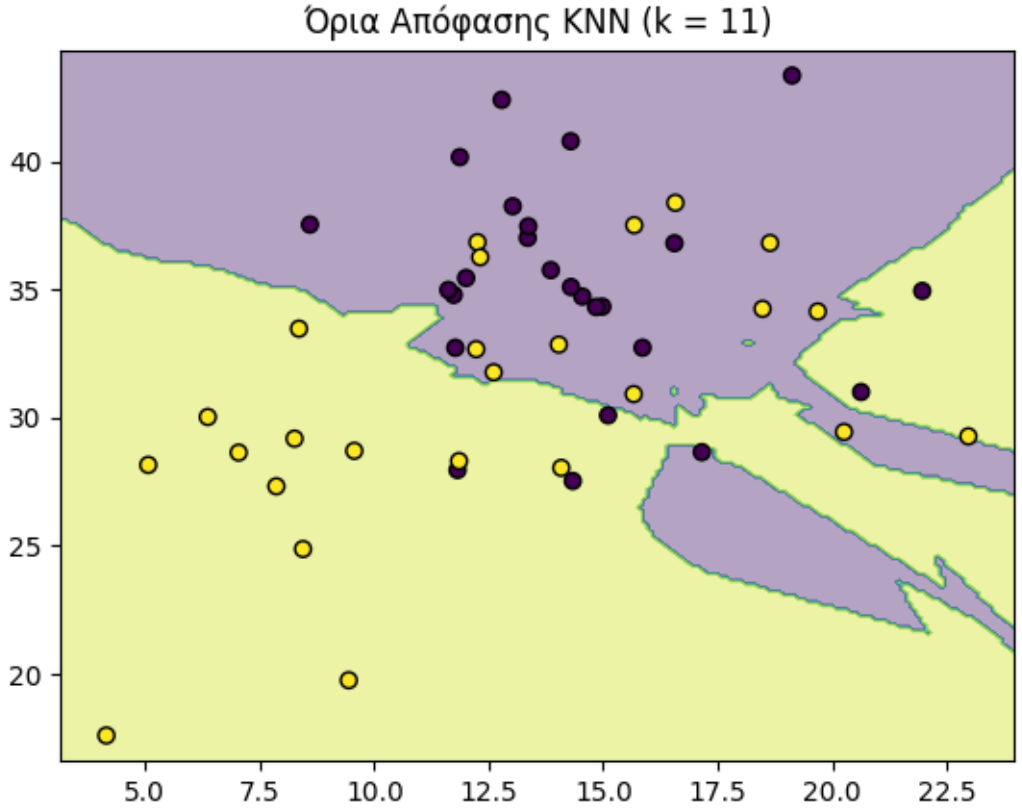
```
# Δημιουργία Grid
xx, yy = np.meshgrid(...)
grid = np.c_[xx.ravel(), yy.ravel()]

# Πρόβλεψη για κάθε σημείο του Grid
grid_labels = np.argmax(predict(..., best_k), axis=1)

# Contour Plot
plt.contourf(xx, yy, Z, alpha=0.4)
```

Decision Boundaries (Αποτέλεσμα)

Το τελικό γράφημα με τις χρωματισμένες περιοχές.
Δείχνει τα μη-γραμμικά όρια που δημιουργεί ο k-NN.
Απεικονίζει τα δεδομένα εκπαίδευσης πάνω στα όρια.



Συμπεράσματα k-NN

- Απλός και διαισθητικός αλγόριθμος.
- Υψηλό υπολογιστικό κόστος στο testing (υπολογίζει αποστάσεις κάθε φορά).
- Η επιλογή του k επηρεάζει σημαντικά το bias-variance trade-off.

Μέρος Δ: Σύγκριση Ταξινομητών

- Στόχος: Ταξινόμηση multiclass δεδομένων (DatasetTV).
- Στρατηγική: Δοκιμή πολλών μοντέλων και Ensemble Learning.
- Μοντέλα: Logistic Regression, RF, GBM, SVM, Stacking.

Προεπεξεργασία (Preprocessing)

- Dataset: Ισορροπημένο (Balanced classes).
- Δεν χρειάστηκε over/under-sampling.
- Split: 75% Training / 25% Validation.
- Correlation: Δεν βρέθηκαν highly correlated features.

Μοντέλο 1: Logistic Regression

- Ρόλος: Baseline γραμμικό μοντέλο.
- Παράμετροι: C (μικρό για regularization), Solver='lbfgs'.
- Απόδοση: Πολύ γρήγορη εκπαίδευση (μερικά δεκάδες δευτερόλεπτα).

Μοντέλο 2: Random Forest (Bagging)

- Λειτουργία: Πολλά δέντρα απόφασης παράλληλα.
- Παράμετροι: n_estimators=200, max_features='sqrt'.
- Πλεονέκτημα: Μειώνει τη διακύμανση (Variance).

Μοντέλο 3: Gradient Boosting

- Λειτουργία: Διαδοχική διόρθωση σφαλμάτων.
- Παράμετροι: Learning rate=0.1, Max depth=4.
- Μειονέκτημα: Πολύ αργό training (περίπου 2 ώρες!).

Μοντέλο 4: Support Vector Classifier (SVC)

- Λειτουργία: Kernel trick (RBF) για μη-γραμμικό διαχωρισμό.
- Παράμετροι: C=5 (Trade-off), Probability=True.
- Παρατήρηση: Απαιτεί Standard Scaler.

Διαδικασία Tuning (GridSearchCV)

- Μεθοδολογία: 3-Fold Cross Validation.
- Metric: F1-weighted (αντί για απλό accuracy).
- Χρήση Pipelines για αποφυγή Data Leakage.

Κώδικας Tuning

Η συνάρτηση αυτοματοποίησης του GridSearchCV:

```
def tune_model(..., return_model=False):
    grid = GridSearchCV(pipeline, param_grid, cv=3,
                        scoring='f1_weighted', n_jobs=-1)
    grid.fit(X_train, y_train)
    return grid.best_estimator_
```

Stacking Classifier (Meta-Learning)

- Συνδυάζει τις προβλέψεις των 4 μοντέλων.
- Base Learners: LR, RF, GBC, SVC.
- Meta Model: Logistic Regression.
- Μέθοδος: 'predict_proba' (Soft voting).

Υλοποίηση Stacking

Δημιουργία του Ensemble Μοντέλου:

```
stacking_clf = StackingClassifier(
    estimators=[('lr', best_lr), ('svc', best_svc)...],
    final_estimator=LogisticRegression(),
    stack_method='predict_proba'
)
```

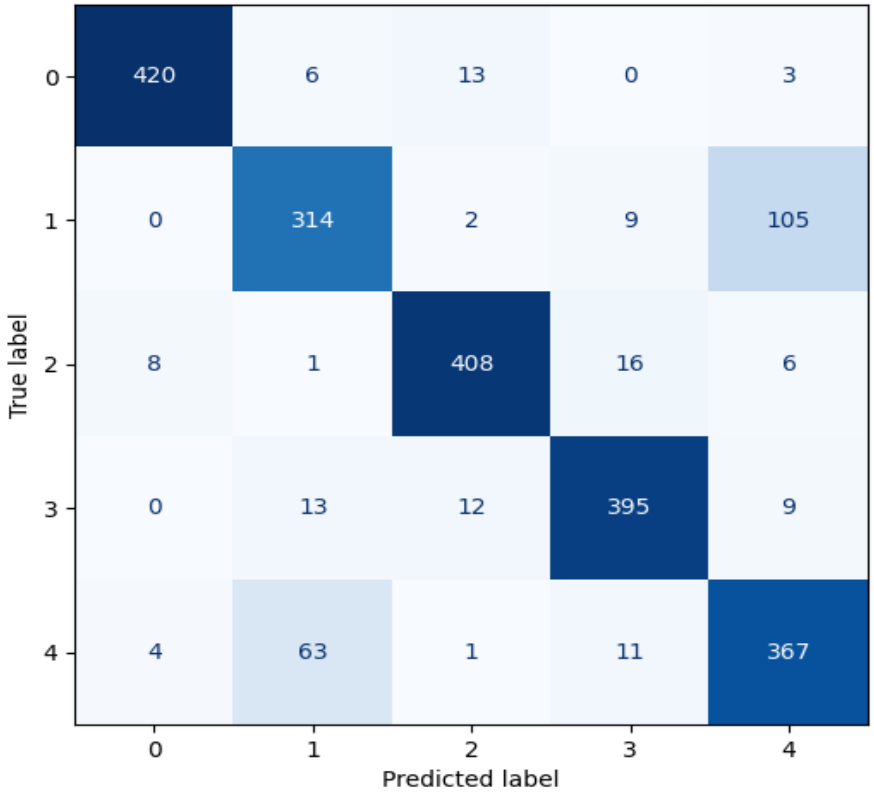
Συνολικά Αποτελέσματα

Πίνακας σύγκρισης μετρικών (Accuracy, F1, etc).
Το Stacking Classifier πέτυχε τα καλύτερα αποτελέσματα.
Ο SVC ήταν πολύ κοντά σε απόδοση.

	accuracy	precision	recall	f1
Logistic Regression	79.28	79.17	79.28	79.18
RandomForestClassifier	82.66	82.52	82.66	82.57
GradientBoostingClassifier	82.02	82.11	82.02	82.02
SVC	86.23	86.33	86.23	86.21
Stacking Classfier	87.10	87.28	87.10	87.13

Confusion Matrix: Το πρόβλημα

Παρατηρήθηκε συστηματική σύγχυση μεταξύ Κλάσης 1 και Κλάσης 4.
Εμφανίζεται σε ΟΛΑ τα μοντέλα (ακόμα και στο Stacking).
Παρακάτω φαίνεται το confusion matrix του Stacking classifier.



Ανάλυση Σφάλματος (Structural)

- Δεν είναι θέμα παραμέτρων (Parameter tuning issue).
- Είναι δομικό πρόβλημα (Structural problem).
- Υπάρχει overlap στο feature space μεταξύ των κλάσεων 1 και 4.

Τελική Επιλογή & Test Set

Επιλέχθηκε ο Stacking Classifier.
Επανεκπαίδευση σε ΟΛΟ το DatasetTV.
Πρόβλεψη στο DatasetTest και αποθήκευση.

```
stacking_clf.fit(X, y)
labelsX = stacking_clf.predict(X_test)
np.save('labelsX.npy', labelsX)
```

Συμπεράσματα

- 1. Το Stacking βελτιώνει οριακά την ακρίβεια.
- 2. Ο SVC είναι η 'χρυσή τομή' απόδοσης/χρόνου.
- 3. Ορισμένες κλάσεις (1 & 4) δεν διαχωρίζονται εύκολα.

Τέλος Παρουσίασης

Ονοματεπώνυμο: Δημόπουλος Μάριος-Γεώργιος, Παπαχριστοδούλου Γεωργία
ΑΕΜ: 10930, 11177